

PREDICTIVE MAINTENANCE

Project Summary & Technical Report

Industrial Machine Failure Prediction System

Classification Pipeline | Streamlit Deployment | LLM-Powered Maintenance Assistant

Dataset	Predictive Maintenance Dataset (AI4I 2020)
Problem Type	Multiclass Classification (7 classes)
ML Framework	scikit-learn, XGBoost, LightGBM, CatBoost
Deployment	Streamlit + OpenAI GPT-4o-mini
Prepared	March 2025

1 Project Overview

Business Problem

Unexpected machine failures in industrial manufacturing environments lead to costly downtime, unplanned maintenance, and production losses. Traditional reactive or calendar-based maintenance strategies are inefficient, often resulting in either excessive downtime for unnecessary checks or catastrophic failures when issues go undetected.

Project Objective

Develop an end-to-end machine learning system that predicts industrial machine failures — and identifies the specific **failure type** — from real-time sensor readings. The system is deployed as an interactive dashboard with an LLM-powered maintenance assistant that translates model predictions into actionable recommendations for factory operators.

Why Predictive Maintenance?

- Reduces unplanned downtime by 30-50% through early failure detection.
- Lowers maintenance costs by replacing reactive repairs with targeted interventions.
- Extends equipment lifespan by preventing cascading damage from minor faults.
- Improves worker safety by flagging dangerous failure conditions before they occur.

2 Dataset Description

The **AI4I 2020 Predictive Maintenance Dataset** (Kaggle) is a synthetically generated industrial dataset widely used as a benchmark for predictive maintenance research. It simulates sensor readings from industrial machines under various operating conditions.

Dataset Summary

Property	Value
Total Samples	10,000
Original Features	14 (including identifiers & failure flags)
Input Features (final)	7 (5 numeric + 2 one-hot encoded Type)
Target Classes	7 (No Failure, TWF, HDF, PWF, OSF, RNF, Multiple Failures)
Problem Type	Multiclass Classification
Missing Values	None (complete case dataset)
Duplicate Rows	None

Input Features

- **Air temperature [K]** — ambient air temperature (295.3 - 304.5 K)
- **Process temperature [K]** — process internal temperature (305.7 - 313.8 K)
- **Rotational speed [rpm]** — shaft rotation speed (1,168 - 2,886 rpm)
- **Torque [Nm]** — applied torque (3.8 - 76.6 Nm)

- **Tool wear [min]** — cumulative tool usage (0 - 253 min)
- **Type** — machine type (L / M / H), one-hot encoded

Target Variable

The target **Reason of Failure** is derived from five binary failure-mode flags (TWF, HDF, PWF, OSF, RNF). If no flag is active → *No Failure*. If exactly one flag fires → the corresponding failure type. If multiple flags fire simultaneously → *Multiple Failures*.

Class Distribution (Original)

Failure Type	Count	Percentage
No Failure	9,652	96.52%
HDF (Heat Dissipation Failure)	106	1.06%
PWF (Power Failure)	80	0.80%
OSF (Overstrain Failure)	78	0.78%
TWF (Tool Wear Failure)	42	0.42%
Multiple Failures	24	0.24%
RNF (Random Failure)	18	0.18%

Note: The dataset is highly imbalanced — No Failure accounts for >96% of samples. This imbalance makes raw accuracy an unreliable metric; per-class precision, recall, and F1-score are essential for evaluating rare-failure detection.

3 Data Analysis & Preprocessing

Exploratory Data Analysis — Key Findings

- **Target imbalance** — No Failure dominates (96.5%). Minority classes (RNF, Multiple Failures, TWF) have <50 samples each.
- **Feature relationships** — Rotational speed and torque show strong negative correlation ($r \approx -0.87$), consistent with the power equation $P = \tau \cdot \omega$.
- **Failure signatures** — Failure points cluster at extremes of the torque-speed relationship. Tool wear is notably elevated for TWF and OSF.
- **Machine types** — Type L is most common (~58%), followed by M (~30%) and H (~12%).
- **No missing values or duplicates** — dataset required no imputation.

Outlier Handling

Z-score analysis identified 164 outliers in Rotational speed ($|z| > 3$) and 25 in Torque. **Outliers were retained** because investigation showed they coincide with actual failure conditions — particularly PWF and Multiple Failures — indicating they carry discriminative signal rather than noise.

Preprocessing Pipeline

- **Target construction** — Built `Reason of Failure` (7 classes) from the five raw failure-mode flags. Dropped `UDI`, `Product ID`, `Machine failure`, and the five flag columns.
- **Encoding** — Machine Type (L/M/H) one-hot encoded with drop_first: `Type\_2` (M) and `Type\_3` (H).

- **Train/Test split** — Stratified 80/20 split (8,000 train / 2,000 test) **before** scaling and balancing to prevent data leakage.
- **Scaling** — StandardScaler fitted on training set only, applied to both train and test.
- **Class imbalance handling (two-stage):**
 - Stage 1 — **RandomOverSampler**: Oversampled minority classes (<50 samples) to a minimum of 50 to provide sufficient neighbours for SMOTE.
 - Stage 2 — **SMOTE** (k=5): Balanced all 7 classes to 7,722 samples each, producing 54,054 training rows.

Caution: The rarest classes (RNF: 14, Multiple Failures: 19 real samples) have limited real-world diversity even after SMOTE augmentation — this constraint is reflected in their per-class scores.

4 Machine Learning Approach

Models Trained

Five classifiers were trained on the preprocessed, balanced dataset:

Model	Type	Imbalance Strategy	Hyperparameters
Logistic Regression	Linear	class_weight='balanced'	max_iter=1000
Random Forest	Ensemble (Bagging)	class_weight='balanced'	n_estimators=100
XGBoost	Gradient Boosting	Sample weights (balanced)	eval_metric='mlogloss'
LightGBM	Gradient Boosting	class_weight='balanced'	n_estimators=100
CatBoost	Gradient Boosting	Class weights (balanced)	iterations=100, verbose=0

Training Strategy

- All models trained on the SMOTE-balanced training set (54,054 rows × 7 features).
- Target encoded with **LabelEncoder** for consistent class ordering across models.
- Feature names cleaned (removed [] <) for XGBoost/LightGBM compatibility.
- Evaluation on the **original, untouched test set** (2,000 rows, real-world distribution).
- Metrics: Accuracy, Precision (weighted), Recall (weighted), F1-score (weighted), Macro F1.
- No hyperparameter tuning (defaults + class balancing) — focused on baseline comparison.

5 Model Performance

Overall Comparison (Weighted Averages)

Model	Accuracy	Precision	Recall	F1-Score
XGBoost	0.9770	0.9837	0.9770	0.9797
LightGBM	0.9745	0.9819	0.9745	0.9777
Random Forest	0.9735	0.9769	0.9735	0.9744
CatBoost	0.9485	0.9754	0.9485	0.9601

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.6570	0.9717	0.6570	0.7724

Best Model: XGBoost — Achieves the highest overall accuracy (97.70%), precision (98.37%), recall (97.70%), and F1-score (97.97%). Its gradient-boosted architecture excels at learning complex nonlinear decision boundaries between normal operation and the six distinct failure modes, even with limited minority-class data.

Per-Class Performance — XGBoost (Best Model)

Failure Type	Precision	Recall	F1-Score	Support
HDF	0.88	1.00	0.93	21
Multiple Failures	1.00	0.40	0.57	5
No Failure	0.99	0.98	0.99	1,930
OSF	0.75	0.94	0.83	16
PWF	0.79	0.94	0.86	16
RNF	0.00	0.00	0.00	4
TWF	0.05	0.12	0.07	8

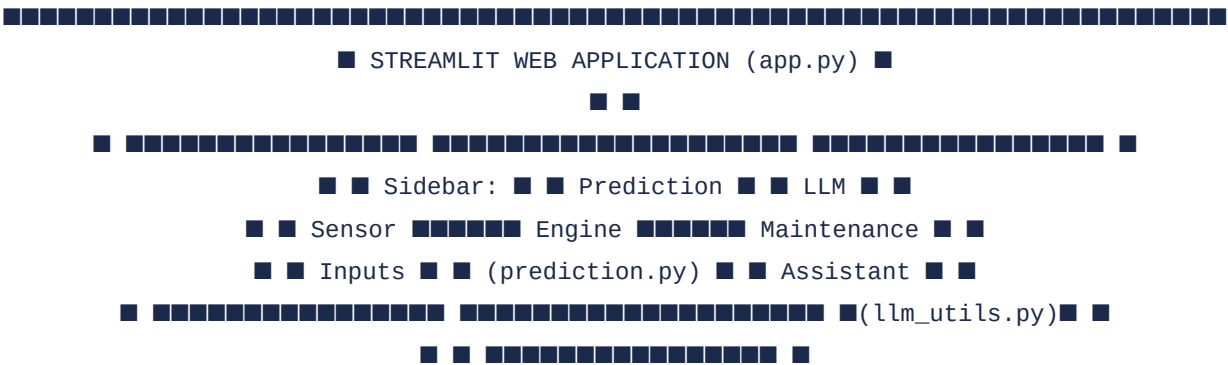
Key Observations:

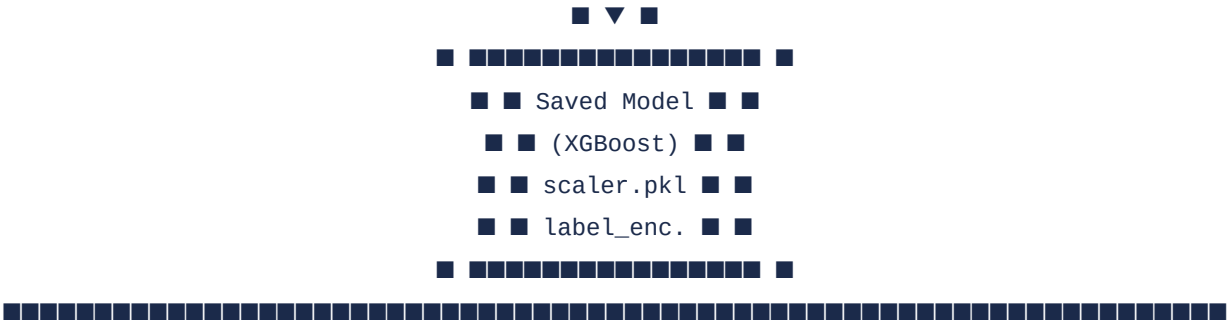
- **No Failure** (99% F1): Nearly perfect — the system reliably identifies normal operation, minimizing false alarms.
- **HDF** (93% F1) and **PWF** (86% F1): Strong performance for common failure modes — the model captures their distinct sensor signatures well.
- **OSF** (83% F1): Good detection, with 94% recall capturing most overstrain events.
- **RNF** (0% F1) and **TWF** (7% F1): Poor performance due to extreme data scarcity (only 4 and 8 real test samples respectively). These classes are fundamentally limited by the dataset; a dedicated data collection effort is required for improvement.
- **Multiple Failures** (57% F1): 100% precision (no false positives) but only 40% recall — the model is conservative about flagging this rare class.

6 Deployment System

Application Architecture

The system is deployed as a **Streamlit** web application with a three-layer architecture:





User Workflow

- **Input:** Operator enters machine readings via sidebar sliders — Machine Type, Air Temp, Process Temp, Rotational Speed, Torque, Tool Wear.
- **Prediction:** The input is scaled (StandardScaler), one-hot encoded, and fed to the saved XGBoost model. Returns failure type + confidence score.
- **LLM Recommendation:** The prediction and sensor readings are sent to OpenAI GPT-4o-mini which generates a context-aware maintenance recommendation.
- **Output:** Prediction displayed with colour-coded alert (green = no failure, red = failure) plus the prose recommendation below.

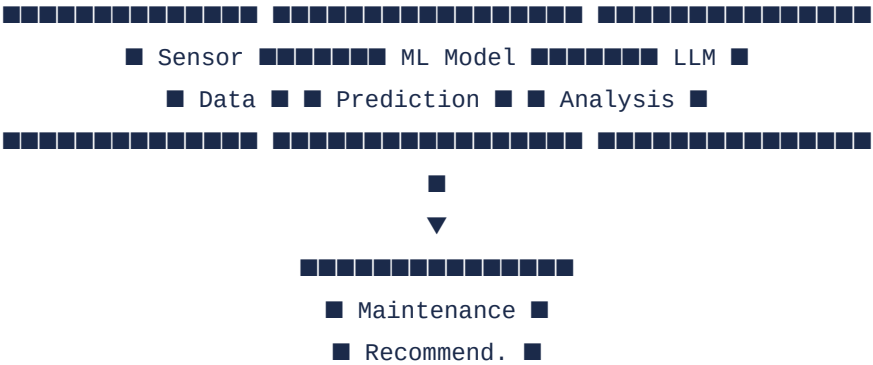
Technologies Used

Component	Technology
Web Framework	Streamlit
ML Backend	scikit-learn, XGBoost, joblib
LLM Integration	OpenAI GPT-4o-mini (REST API)
Data Handling	pandas, NumPy
Serialisation	joblib (model, scaler, label encoder)
Environment	Python 3.11, Windows 11

7 Generative AI Integration

The system integrates **OpenAI GPT-4o-mini** as an intelligent maintenance assistant that converts raw model predictions into actionable, human-readable recommendations for factory floor operators.

System Pipeline





Integration Details

Aspect	Detail
LLM Model	GPT-4o-mini (OpenAI)
Temperature	0.3 (low randomness for consistent recommendations)
Max Tokens	300
Prompt Strategy	Sensor readings + prediction → structured task description
Fallback	7 hardcoded recommendation templates (one per failure type)
Provider Flexibility	Supports both OpenAI and Anthropic (configurable via env)

Input Provided to the LLM

The prompt is built from:

- All sensor readings (machine type, temperatures, speed, torque, tool wear).
- The model's prediction (failure type) and confidence probability.
- A task instruction: explain possible cause, immediate action, maintenance recommendation, and prevention tips.

Generated Output

A concise (<150 words) maintenance recommendation including:

- Root cause analysis linked to specific sensor readings.
- Immediate operator actions (e.g., shut down, inspect subsystem X).
- Preventive measures to avoid recurrence.

How LLM Improves the System

- Translates opaque probability scores into **domain-specific, actionable advice**.
- Adapts recommendations to **specific sensor context** (e.g., high torque + low speed → overstrain inspection).
- Provides a **natural-language bridge** between data scientists and factory operators.
- Graceful **fallback templates** ensure system works even without API connectivity.

8 Challenges & Future Improvements

Current Limitations

- **Extreme class imbalance:** RNF (only 18 samples), Multiple Failures (24), and TWF (42) have insufficient real-world examples. SMOTE cannot create truly representative synthetic data from such small pools.
- **Rare-class detection gap:** RNF and TWF F1-scores of 0.00 and 0.07 mean these failures would be missed in production — a safety risk.

- **No hyperparameter tuning:** Models used default parameters (except class balancing). Grid search / Optuna could yield further gains.
- **Synthetic dataset:** AI4I 2020 is generated data — real-world sensor noise, drift, and novel failure modes are not represented.
- **LLM dependency:** Recommendations require API connectivity and incur per-call costs. Fallback templates are static and less context-aware.

Dataset Challenges

- Only 10,000 samples — insufficient for deep learning approaches.
- No time-series component — each row is an independent snapshot rather than a temporal sequence.
- Synthetic nature raises questions about transferability to real industrial environments.

Possible Improvements

- **Collect real-world failure data** — especially for RNF, TWF, and Multiple Failures — to populate the rare classes with genuine examples.
- **Hyperparameter optimisation** — perform Bayesian search (Optuna) with macro F1 as the objective to balance rare-class performance.
- **Anomaly detection** — add a one-class SVM or autoencoder to flag novel failure signatures not seen during training.
- **Temporal modeling** — if sequence data becomes available, LSTM or Transformer models could capture degradation trends over time.
- **Ensemble approach** — combine XGBoost (general performance) with a rare-class-specialised model for production deployment.
- **Local LLM deployment** — replace cloud API with a smaller on-premise model (e.g., Llama 3 8B quantised) to eliminate latency and privacy concerns.

9 Final Project Summary

This project delivers a complete, end-to-end predictive maintenance system for industrial machinery, from data preprocessing through to LLM-enhanced deployment. The key achievements are:

Best ML Model: XGBoost — 97.7% accuracy, 97.97% weighted F1-score. Achieves near-perfect detection of the dominant No Failure class (99% F1) and strong performance on HDF (93% F1), PWF (86% F1), and OSF (83% F1).

Streamlit Deployment: A user-friendly web interface where operators enter sensor readings and receive instant failure predictions with colour-coded confidence alerts.

LLM Maintenance Assistant: GPT-4o-mini integration transforms ML predictions into context-aware, actionable maintenance recommendations — bridging the gap between AI outputs and real-world operator decisions.

Full ML Pipeline: Complete, reproducible pipeline: target engineering → EDA → two-stage SMOTE balancing → 5-model comparison → best-model serialisation.

Technical Value: Demonstrates a production-ready industrial AI system combining classical gradient boosting with modern LLM integration — achieving 97.7% predictive accuracy while providing explainable,

actionable maintenance guidance.

Real-World Impact: This system can reduce unplanned downtime by providing early warning of six distinct failure modes, enabling targeted preventive maintenance, extending equipment lifespan, and improving factory floor safety. The LLM assistant ensures that even non-specialist operators can act on ML predictions with confidence.